Data mining practicum 1

Berend Dekker (5802954), Max van Huffelen (1714236), Gerard van Schie (6585442)

September 2022

1 Short introduction of the data

For this assignment, the Eclipse bug dataset will be used. This dataset contains information about the (number of) bugs reported during both the six months before and after the release of the Eclipse software in each package of the software, for versions including 2.0 and 3.0. Furthermore, for each package a number of other parameters is included. These parameters describe a number of metrics of the packages, such as their number of methods, classes and lines of codes.

A classification tree model will be trained on the data of version 2.0. This model will then be tested using the data from version 3.0, where it will predict if bugs are still present in the new version. Here, the data has been edited slightly in order to be able to perform binary classification. Rather than listing the number of bugs reported, it now lists whether bugs have been found (indicated with '1') or not (indicated with '0'). Bugs have been reported in about half the packages.

2 Classification tree

Figure 1 shows the first two splits of a single classification tree. It has been generated by computing the impurity reduction of each possible split for each attribute and selecting the split with the highest impurity reduction. In this section we will go over the splits and compare it with the findings in research [ZPZ].

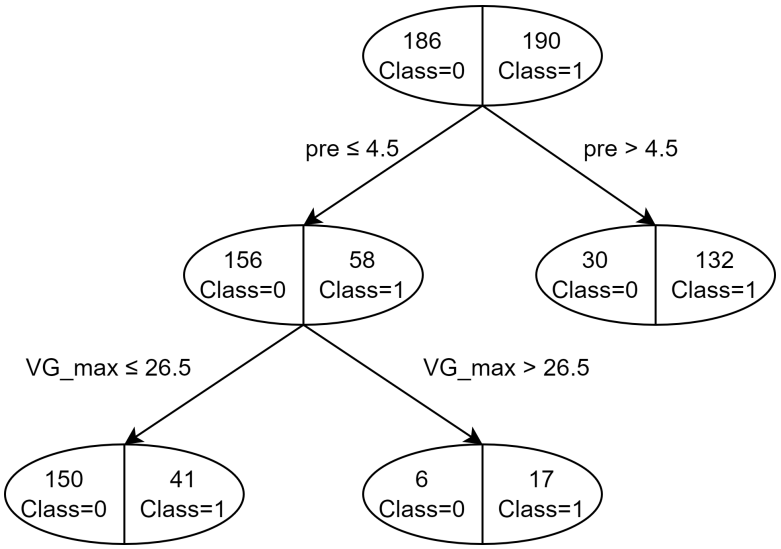

Figure 1: Visual representation of the two first splits in the classification tree. The top node is the root containing all items in the test set. The connections describe the split attribute and the value it splits the items on. Class '1' indicates bugs have been found, class '0' indicates no bugs were found in the six months after release as described in section 1.

2.1 First Split

The first split performed is on the attribute 'pre'. This attribute denotes the number of bugs that have been reported in the six months before release. It makes sense that this splitting on this attribute helps predict if there will still be bugs post-release. After all, if a piece of software is particularly bug-prone, it is likely that new bugs will be reported after release, or that some of the old bugs still remain.

As expected, this behaviour is indeed showcased by the first split of the classification tree. From an almost equal split of item classes in the root node, a lot of items with bugs post-release (class '1') are moved to the

right node if 'pre' was greater than 4.5. Similarly, for the cases on the left-hand side there are relatively many packages that had no bugs reported post-release.

In the aforementioned research[ZPZ] in table 3 the correlation between having bugs in the post release from the number of bugs in the pre-release has the highest value in the column with 0.786. This indicates that there is a very strong correlation which further increases the confidence in this first split.

2.2 Second Split

The parameter 'VG_max' describes the maximum McCabe cyclomatic complexity[McC76] of the entire package. This measure gives a complexity value of the program flow graph, a higher value means a more complicated program flow graph. An increase in complexity of the package code would be expected to increase the likelihood of reported bugs after the release.

Thus, the 'VG_max' attribute can be expected to be a good attribute to split on. The chosen value here is 26.5. Indeed, it can be seen that modules with a complexity above this value have a significantly higher likelihood of containing bugs than those with a lower value.

With a value of 0.430 this attribute is not the second highest value in the correlations. That would have been 'TLOC_sum' with a correlation value of 0.559. However, there is still a correlation between 'VG_max' and whether there are bugs reported after the release. In the root node the best split could have already moved most of the items classifiable by 'TLOC_sum' to the right side which would explain why the algorithm chose to split on attribute 'VG_max' instead.

3 Results

3.1 Single Tree

This is the test for a single tree, with the parameters that are specified in the practicum. The chance of a success for a single tree is equal to 73.52% and the chance of failure is equal to 26.48%.

Accuracy	0.7352
Precision	0.7522
Recall	0.7414

		Prediction outcome		
		p	**n**	**total**
actual value	**p′**	258	90	348
	n′	85	228	313
	total	343	318	661

Figure 2: Accuracy, precision, recall and the confusion matrix

3.2 Bagging

This is the test for bagging, with the parameters that are specified in the practicum, the chance of a success for bagging is equal to 77.16% and the chance of failure is equal to 22.84%.

Accuracy	0.7716
Precision	0.7532
Recall	0.8420

		Prediction outcome		
		p	**n**	**total**
actual value	**p′**	293	55	348
	n′	96	217	313
	total	389	272	661

Figure 3: Accuracy, precision, recall and the confusion matrix

3.3 Random Forest

This is the test for Random Forest, with the parameters that are specified in the practicum the chance for success for a Random Forest is equal to 76.10% and the chance for failure is 23.90%.

Accuracy	0.7610
Precision	0.7582
Recall	0.8158

		Prediction outcome		
		p	**n**	**total**
actual value	**p′**	279	69	348
	n′	89	224	313
	total	368	293	661

Figure 4: Accuracy, precision, recall and the confusion matrix

4 Discussion about results

4.1 Statistical Method

It has been decided to model the data as a Bernoulli experiment in order to test whether the difference in accuracy between the Single Tree, Bagging and Random Forest is due to noise or if the differences are statistically significant. However, Bernoulli experiments only work on binary variables, As such, in order to model it correctly, it was decided to group the false negatives and false positives into one category, 'Failures', and true positives and true negatives into the other category, 'Successes'. Thus, in the model the chance of x successes with N classifications is modeled as:

$$\text{P(Successes=x)} = \binom{n}{x} \cdot \text{P(Success)}^{x} \cdot \text{P(Failure)}^{(N-x)} \tag{1}$$

Where $P(Success)$ is the chance that an element is classified successfully, and $P(Failure) = 1 - P(Success)$ the chance that it is classified incorrectly. Approximate values for these probabilities are obtained by dividing the total amount of successes and failures by the total amount of classifications, respectively. Then, these probabilities can be used to find the chances that one model performs as good or better than another.

If a model M_1 is to be compared to another model, M_2, the models may be used to classify a test set of size N. Let S_1 and S_2 be the number of inputs classified correctly by M_1 and M_2, respectively. If, for example, model M_1 performs better than the other (i.e. $S_1 > S_2$), it is desirable to know if this model really is better, or if this is caused by noise. To do this, the (true) probability of M_1 correctly classifying an input is approximated as $p_1 = \frac{S_1}{N}$. Then, the following equation can be used to find the probability that M_1 classifies as many or more inputs correctly than M_2:

$$P(S_1 \geq S_2) = \sum_{i=S_2}^{N} \binom{N}{i} \cdot p_1^i \cdot (1-p_1)^{N-i} \tag{2}$$

Similarly, in order to calculate the chance that M_1 performs as bad or worse than M_2, the next equation can be used:

$$P(S_1 \leq S_2) = \sum_{i=0}^{S_2} \binom{N}{i} \cdot p_1^i \cdot (1-p_1)^{N-i} \tag{3}$$

If either of these probabilities is at least 97.5%, it can be concluded, with a p-value of 0.05, that M_1 has a higher or lower probability of correctly classifying an input than M_2, respectively.

4.2 Statistical Results Analysis

In order to analyse which of the methods among Single Tree, Bagging and Random Forest provide the best results, a statistical analysis has been performed according to the method provided in subsection 4.1. The

results of these analyses are shown in Table 1 and Table 2. These tables show the probabilities that the model listed on the vertical axis is better (Table 1 or worse (Table 2 than the model listed on the horizontal axis.

Better	Single Tree	Bagging	Random Forest
Single Tree	-	0.01790	0.07136
Bagging	0.9875	-	0.7586
Random Forest	0.94368	0.27904	-

Table 1: The chances one model is better than another

Worse	Single Tree	Bagging	Random Forest
Single Tree	-	0.98575	0.94013
Bagging	0.01574	-	0.27091
Random Forest	0.06715	0.75106	-

Table 2: The chances one model is worse than another

The data shows that the only error probabilities higher then 0.975 is that the Single Tree method performs worse than Bagging and that the Bagging yields better results than a Single Tree. It is therefore likely that Bagging is significantly better than a Single Tree, with a chance of at least 95%. Unfortunately, (the size of) the data does not support conclusions about the Random Forest model in comparison to the other models. In order to draw such conclusions, and thus conclude which of the three models is preferred, more data would be needed. While the current results might seem to indicate that, for example, Random Forest performs better than Single Tree, we cannot determine with high enough probability whether this is due to statistical noise or a real difference in performance.

References

[McC76] Thomas J. McCabe. "A Complexity Measure". In: *IEEE TRANSACTIONS ON SOFTWARE ENGINEERING* SE-2.4 (Dec. 1976), pp. 308–320. URL: `http://www.literateprogramming.com/mccabe.pdf`.

[ZPZ] Thomas Zimmermann, Rahul Premray, and Andreas Zeller. *Predicting Defects for Eclipse*. URL: `https://www.st.cs.uni-saarland.de/softevo/bug-data/eclipse/promise2007-dataset-20a.pdf`.